



网络安全创新创业实践课实验报告

SM3 软件实现与 SIMD 优化

ARM64 与 x86 AVX2/AVX-512 混合寄存器实现

第 41 组

2026 年 7 月 30 日

小组成员及分工

本实验由小组成员共同完成。实验按照处理器架构和工作内容，划分为公共标量实现、ARM64 优化、x86 SIMD 优化以及测试分析四个部分。各成员在完成所负责模块的基础上，共同进行代码审查、结果核验和实验报告整理。

表 1: 小组成员及分工

姓名	学号	班级	负责模块	对应章节
姜良萍（组长）	202300460098	网安 1 班	负责实验总体方案设计与工程组织；整理 SM3 标量参考实现和公共接口；管理 Git 仓库、实验目录、编译脚本与版本合并；协调 ARM64 与 x86 两条实现路线；负责全文整合、格式统一、实验结论总结及最终复核。	第 1 章；第 2.1、2.4 节；第 3.1、3.2 节；第 4.1、4.2 节；第 6 章
李雨晴	202300460025	网安 2 班	负责 x86 平台 SM3 软件实现与优化；实现 AVX2 八路并行、AVX-512 十六路并行和不同长度消息的 lane 掩码调度；实现运行时 CPU 特性检测和后端自动分发；完成标准向量、随机差分、OpenSSL 对照、Sanitizer、反汇编检查及 x86 性能实验。	第 2.2 节；第 3.3–3.5 节；第 4.3–4.6 节；第 5.1–5.7 节
王奕文	202300460056	密码 1 班	负责 ARM64 平台 SM3 软件实现与优化；分析 ARM64 通用寄存器和 NEON 寄存器的混合使用方式；实现 NEON 优化版本与 ARM64 手写汇编版本；研究消息扩展窗口、循环展开、寄存器分配、访存减少和指令调度方法。	第 2.3 节；第 3.6 节；第 4.7 节；第 5.8 节
曾静雯	202300460045	网安 1 班	负责跨平台正确性验证与性能测试组织；整理不同消息长度下的吞吐率、单位字节时间和加速比；执行多轮重复测试并统计中位数；整理实验截图、性能表格和结果图；协助 ARM64 与 x86 实验结果的统一口径比较和报告分析。	第 2.4 节；第 4.6 节；第 5.2–5.8 节；协助第 6 章结果分析

目录

1	实验目的	1
2	实验环境	1
2.1	x86 实验环境	1
2.2	ARM64 实验环境	2
3	实验原理	4
3.1	SM3 算法总体结构	4
3.2	SM3 轮函数	4
3.3	x86 SIMD 优化原理	6
3.4	ARM64 优化原理	9
4	实验设计与实现	13
4.1	总体设计	13
4.2	标量参考实现	13
4.3	AVX2 八路并行实现	14
4.4	AVX-512 十六路并行实现	16
4.5	运行时指令集分发	17
4.6	ARM64 实现	18
5	实验结果与分析	24
5.1	x86 实验结果与分析	24
5.2	ARM64 实验结果与分析	31
6	总结	38

1 实验目的

SM3 是我国自主设计的密码杂凑算法，能够将任意长度消息映射为 256 bit 摘要，广泛应用于数字签名、消息认证、证书系统和国产密码协议中。其压缩函数包含大量 32 bit 模加、循环移位、异或和布尔运算，具有较强的计算密集型特征，因此适合利用现代处理器的通用寄存器和 SIMD 寄存器进行并行优化。

本实验旨在深入理解 SM3 的消息填充、消息扩展、轮函数和迭代压缩过程，首先完成能够处理任意长度消息的标量参考实现，并以此作为后续优化版本的正确性基准。在此基础上，针对 SM3 单条消息轮间状态依赖较强、难以直接将连续轮次映射到不同 SIMD lane 并行执行的问题，采用 Multi-Buffer 多消息并行方法，将多条相互独立的消息分别映射到不同向量通道中，从而提高批量消息处理吞吐率。

在 x86-64 平台上，本实验分别实现基于 AVX2 的八路并行版本和基于 AVX-512 的十六路并行版本，使 YMM 或 ZMM 寄存器承担状态字、消息扩展字、布尔函数、循环移位和模加等核心计算，同时由通用寄存器负责消息地址、长度、循环索引、活动掩码和批次调度，从而形成 SIMD 寄存器与通用寄存器混合使用的优化结构。此外，实验还实现运行时 CPU 特性检测，使程序能够按照 AVX-512、AVX2 和标量实现的优先级自动选择当前平台支持的最优后端。

在 ARM64 平台上，实验进一步完成基于 NEON 与通用寄存器混合使用的 SM3 优化实现，并设计手写汇编版本，以研究寄存器分配、消息扩展窗口、循环展开、访存减少和指令调度等底层优化方法。通过 ARM64 和 x86 两条实现路线，形成覆盖两类主流处理器架构的 SM3 软件优化方案。

最后，实验通过标准测试向量、填充边界长度、随机差分、OpenSSL 对照、AddressSanitizer、UndefinedBehaviorSanitizer 和反汇编检查，对不同实现的正确性、内存安全性和指令集使用情况进行验证；同时比较标量版本与优化版本在不同消息长度下的吞吐率、单位字节时间和加速比，分析输入重排、消息填充、活动掩码、轮间依赖和调度开销对实际性能提升的影响。

2 实验环境

2.1 x86 实验环境

x86 部分在 Intel Core i7-13700H 处理器上完成。主机属于 x86-64 架构，支持 AVX、AVX2 和 AVX-VNNI，但不支持 AVX-512 Foundation。因此，本机能够完整运行 AVX2 八路实现，而 AVX-512 版本只能进行编译和静态反汇编验证。

表 2: x86 实验软硬件环境

项目	配置
处理器	13th Gen Intel Core i7-13700H
处理器架构	x86-64
操作系统	Ubuntu 24.04, 运行于 WSL2
WSL 内核	6.6.87.2-microsoft-standard-WSL2
虚拟化环境	Microsoft Hyper-V / WSL2
WSL 可见逻辑处理器	6
CPU 指令集	AVX、AVX2、AVX-VNNI; 不支持 AVX-512F
编译器	GCC/cc, C11 标准
公共优化参数	-O3 -Wall -Wextra -Wpedantic -Wshadow -Wconversion
AVX2 编译参数	-mavx2
AVX-512 编译参数	-mavx512f

2.2 ARM64 实验环境

ARM64 部分在双路鲲鹏 920 服务器上完成。处理器采用 AArch64 架构和小端字节序，共配置 2 个处理器插槽，每个插槽包含 32 个物理核心，总计 64 个在线核心，且每个核心仅包含 1 个硬件线程。处理器最高工作频率为 2.6GHz，CPU 标识信息为 implementer 0x48、part 0xd01，对应鲲鹏 920 使用的 TaiShan v110 处理器核心。

处理器特征列表中包含 `asimd`，编译器预定义宏中包含 `__ARM_NEON`，因此实验平台支持 ARM Advanced SIMD，即 NEON 指令集。ARM64 部分分别测试标准 C 参考实现、NEON intrinsics 混合优化实现和 ARM64 手写汇编实现。性能测试采用单线程方式，使用 `clock_gettime(CLOCK_MONOTONIC)` 记录运行时间，并可通过 `taskset` 将程序固定到指定处理器核心。

表 3: ARM64 实验软硬件环境

项目	配置
处理器	HiSilicon Kunpeng 920, 双路, 每路 32 核, 共 64 个物理核心
CPU 微架构	TaiShan v110, CPU implementer 为 0x48, CPU part 为 0xd01
处理器架构	AArch64, ARMv8, 64 位小端模式
核心与线程配置	2 个处理器插槽, 32 核/插槽, 1 线程/核心, 共 64 个在线核心
处理器频率	最高 2.6 GHz, 最低 200 MHz, 动态加速关闭
操作系统	Ubuntu 22.04.5 LTS
Linux 内核	5.15.0-143-generic
编译器	GCC/cc 11.4.0, 目标平台为 aarch64-linux-gnu
SIMD 扩展	ARM Advanced SIMD/NEON; 处理器特征包含 asimd, 编译器定义 __ARM_NEON
其他指令扩展	AES、PMULL、SHA-1、SHA-2、CRC32、Atomics、FP16、ASIMDHP、ASIMDDP
缓存配置	L1 数据缓存共 4 MiB, L1 指令缓存共 4 MiB, L2 缓存共 32 MiB, L3 缓存共 64 MiB
NUMA 配置	2 个 NUMA 节点; 节点 0 对应 CPU 0-31, 节点 1 对应 CPU 32-63
测试实现	Reference C、ARM64 NEON intrinsics、ARM64 Assembly
测试方式	单线程运行, 可使用 taskset 固定测试核心
性能测试工具	clock_gettime(CLOCK_MONOTONIC)、taskset

3 实验原理

3.1 SM3 算法总体结构

SM3 输入为任意长度比特串，输出为 256 bit 摘要。算法主要包括消息填充、消息分组、消息扩展、压缩函数和链值更新。

设原始消息长度为 l bit。首先在消息后添加一个比特 1，再添加若干个 0，使填充后的长度满足：

$$l + 1 + k \equiv 448 \pmod{512}.$$

最后追加原始消息长度的 64 bit 大端表示，得到若干个 512 bit 消息分组：

$$B^{(0)}, B^{(1)}, \dots, B^{(n-1)}.$$

SM3 初始向量为：

$$IV = (7380166F, 4914B2B9, 172442D7, DA8A0600, \\ A96F30BC, 163138AA, E38DEE4D, B0FB0E4E).$$

每个 512 bit 分组包含 16 个 32 bit 字。首先扩展为 $W_0, \dots, W_{67}$ ：

$$W_j = \begin{cases} M_j, & 0 \leq j \leq 15, \\ P_1(W_{j-16} \oplus W_{j-9} \oplus (W_{j-3} \lll 15)) \oplus (W_{j-13} \lll 7) \oplus W_{j-6}, & 16 \leq j \leq 67. \end{cases}$$

随后生成：

$$W'_j = W_j \oplus W_{j+4}, \quad 0 \leq j \leq 63.$$

其中：

$$P_0(X) = X \oplus (X \lll 9) \oplus (X \lll 17),$$

$$P_1(X) = X \oplus (X \lll 15) \oplus (X \lll 23).$$

3.2 SM3 轮函数

SM3 压缩函数共有 64 轮，每轮维护 8 个 32 bit 状态字：

$$A, B, C, D, E, F, G, H.$$

布尔函数 FF_j 和 GG_j 根据轮数采用不同定义：

$$FF_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z, & 0 \leq j \leq 15, \\ (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z), & 16 \leq j \leq 63, \end{cases}$$

$$GG_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z, & 0 \leq j \leq 15, \\ (X \wedge Y) \vee (\neg X \wedge Z), & 16 \leq j \leq 63. \end{cases}$$

轮常量为：

$$T_j = \begin{cases} 79CC4519, & 0 \leq j \leq 15, \\ 7A879D8A, & 16 \leq j \leq 63. \end{cases}$$

在第 j 轮中，首先计算两个中间变量：

$$SS_1 = ((A \lll 12) + E + (T_j \lll j)) \lll 7,$$

$$SS_2 = SS_1 \oplus (A \lll 12).$$

随后分别计算左右两条数据路径：

$$TT_1 = FF_j(A, B, C) + D + SS_2 + W'_j,$$

$$TT_2 = GG_j(E, F, G) + H + SS_1 + W_j.$$

最后完成状态更新：

$$D = C,$$

$$C = B \lll 9,$$

$$B = A,$$

$$A = TT_1,$$

$$H = G,$$

$$G = F \lll 19,$$

$$F = E,$$

$$E = P_0(TT_2).$$

上述计算过程的数据流如图 1 所示。左侧路径主要通过 FF_j 、 SS_2 和 W'_j 生成新的状态字 A_{j+1} ，右侧路径主要通过 GG_j 、 SS_1 、 W_j 和置换函数 P_0 生成新的状态字 E_{j+1} 。其余状态字则通过赋值或循环移位完成更新。

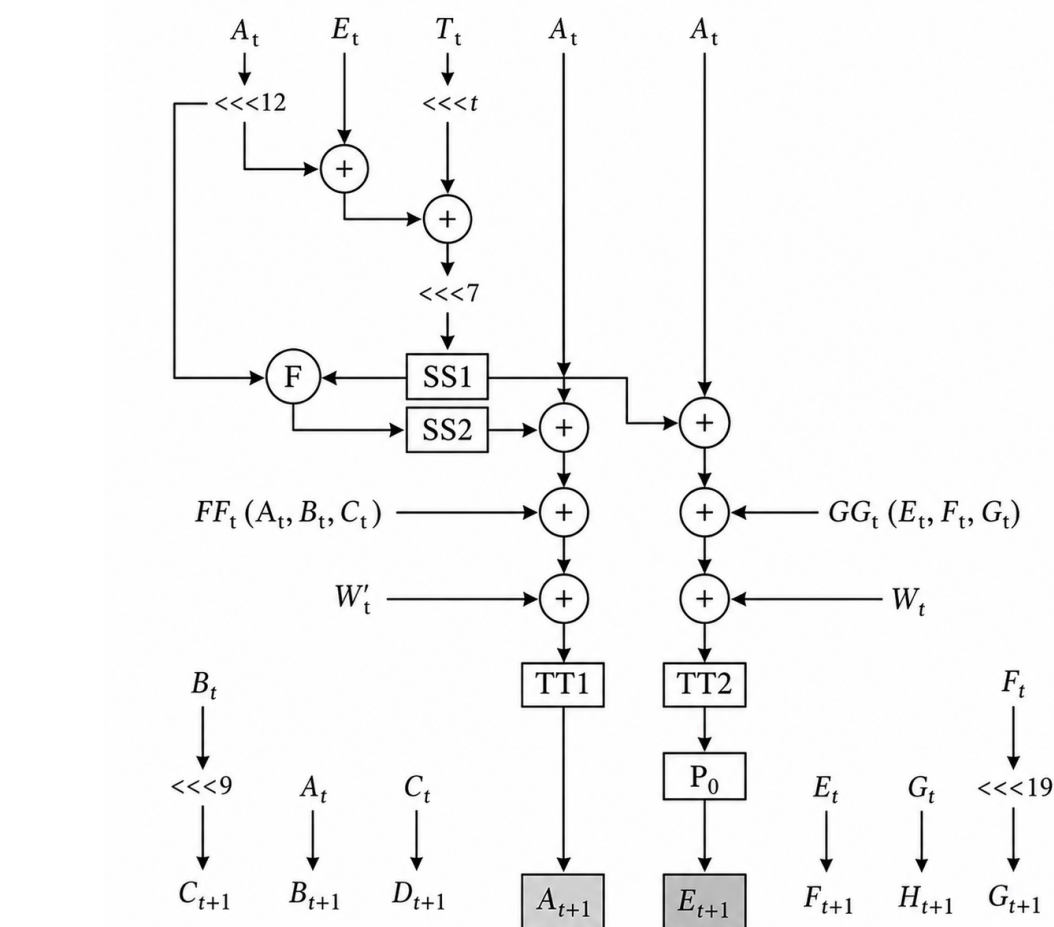


图 1: SM3 单轮压缩函数的数据流

由图 1 可以看出，同一条消息的相邻轮次之间存在严格的数据依赖关系。第 $j + 1$ 轮必须使用第 j 轮更新后的 A 至 H 状态，因此不能简单地将同一条消息的连续轮次映射到不同 SIMD lane 中并行执行。这一特点也是后续采用 Multi-Buffer 多消息并行方案的主要原因。

3.3 x86 SIMD 优化原理

x86 SIMD 优化的核心是将多条独立消息组织为批次，利用宽向量寄存器同时执行消息扩展和压缩运算。本实验主要从多消息数据布局、SIMD 与通用寄存器协同、不同长度消息的掩码处理三个方面进行设计，并分别实现 AVX2 八路并行和 AVX-512 十六路并行版本。

3.3.1 多消息并行的数据布局

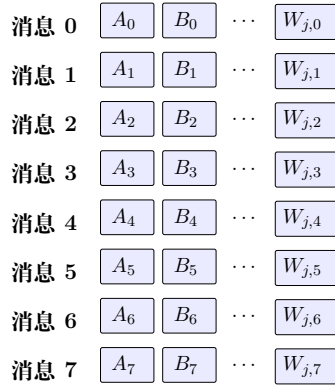
SM3 单条消息的相邻轮次之间存在严格的数据依赖，第 $j + 1$ 轮必须使用第 j 轮更新后的 A 至 H 状态，因此无法将同一条消息的连续轮次直接映射到不同 SIMD lane

中并行执行。

为发挥宽向量寄存器的计算能力，本实验采用 Multi-Buffer 多消息并行方法，将多条相互独立的消息组成一个批次，并将各条消息在相同轮次下的状态字和消息扩展字分别映射到不同 lane。这样，各 lane 之间不存在数据依赖，可以同时执行相同的模加、异或、循环移位和布尔函数运算。

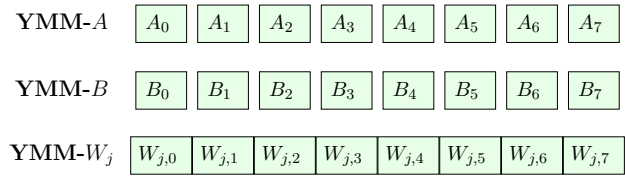
原始数据通常按照消息进行组织，即一条消息的状态字和消息字在逻辑上属于同一组。进入 SIMD 压缩函数前，程序需要将不同消息中相同位置的状态字纵向聚合到同一个向量寄存器中。该 AoS 到 SoA 的逻辑数据布局转换如图 2 所示。

AoS 逻辑布局：按消息组织



SoA 向量布局：按状态字纵向打包

AoS → SoA
纵向打包



AVX2: 一个 YMM 寄存器包含 8×32 bit lane, 可并行处理 8 条消息

AVX-512 沿用相同布局:

ZMM-A = $[A_0 | A_1 | \dots | A_{15}]$, 共 16 个 32 bit lane

图 2: x86 Multi-Buffer SM3 的 AoS-SoA 逻辑数据布局

如图 2 所示，AVX2 的一个 `__m256i` 寄存器由 8 个 32 bit lane 组成，因此可以同时保存 8 条消息的同一个状态字。例如，状态字 A 在向量寄存器中的布局为：

$$A_{\text{vec}} = (A_0, A_1, A_2, A_3, A_4, A_5, A_6, A_7),$$

其中， A_i 表示第 i 条消息在当前轮次中的状态字 A 。状态字 B 至 H 均采用相同的纵向组织方式。消息扩展字也按照相同形式打包：

$$W_j^{\text{vec}} = (W_{j,0}, W_{j,1}, \dots, W_{j,7}),$$

其中， $W_{j,i}$ 表示第 i 条消息的第 j 个消息扩展字。因此，一条 AVX2 模加、异或或移位指令，可以同时完成 8 条消息在相同轮次下的对应运算。

AVX-512 版本沿用相同的数据组织方式，但将向量宽度由 256 bit 扩展为 512 bit。一个 ZMM 寄存器可以容纳 16 个 32 bit lane，其状态字 A 的布局为：

$$A_{\text{vec512}} = (A_0, A_1, \dots, A_{15}),$$

从而将批量并行宽度由 AVX2 的 8 路扩展为 AVX-512 的 16 路。

这种数据布局转换本质上是将按消息组织的 AoS 逻辑形式，转换为按状态字聚合的 SoA 向量形式。该过程会引入一定的输入装载和数据重排开销，但能够使消息扩展、布尔函数、循环移位、模加和状态更新等计算密集型操作充分利用 SIMD 指令。随着消息长度增加，数据重排和批次调度开销在总计算量中的占比逐渐降低，SIMD 并行带来的吞吐率提升也会更加稳定。

3.3.2 SIMD 寄存器与通用寄存器协同

在 x86 优化实现中，SIMD 寄存器主要承担计算密集型的数据路径。AVX2 版本使用 YMM 寄存器，AVX-512 版本使用 ZMM 寄存器，用于保存多路 A 至 H 状态、消息扩展字 W_j 和 W'_j ，并完成 32 bit 模加、异或、布尔函数、循环移位以及 P_0 、 P_1 等核心运算。

通用寄存器则主要承担控制和调度任务，包括保存输入输出地址、消息长度、分组位置和循环索引，并负责填充分组构造、批次切换及尾部消息处理。两类寄存器相互配合，使计算密集型操作在宽向量寄存器中并行执行，而消息管理和控制逻辑仍由通用寄存器完成，从而形成 SIMD 数据通路与标量控制通路相结合的混合优化结构。

后续反汇编结果中可以同时观察到 `rax`、`rbx`、`rsi`、`rdi`、`r8` 等通用寄存器，以及 `ymm` 或 `zmm` 向量寄存器，说明最终生成的机器代码确实采用了上述混合寄存器设计。

3.3.3 不同长度消息的掩码处理

同一批次中的多条消息长度可能不同，因此不同 lane 完成计算的时间也可能不同。如果某个 lane 已经处理完全部分组，后续迭代就不能继续修改该 lane 的链值。

在 AVX2 版本中，程序为 8 个 lane 构造全零或全一的向量掩码，并通过：

```
1 _mm256_blendv_epi8(  
2     old_state,  
3     new_state,  
4     active_mask  
5 )
```

选择性提交新的状态。仍处于活动状态的 lane 使用新计算结果，已经完成的 lane 则保留原状态。

AVX-512 提供独立的掩码寄存器，因此程序使用 `__mmask16` 表示 16 个 lane 的活动状态，并通过：

```
1 _mm512_mask_mov_epi32(  
2     old_state,  
3     active_mask,  
4     new_state  
5 )
```

完成按 lane 更新。该机制使 SIMD 实现不仅能够处理相同长度的批量消息，也能够正确处理同一批次中长度不同的消息。

3.4 ARM64 优化原理

ARM64 部分主要面向鲲鹏 920 平台进行优化。与 x86 Multi-Buffer 实现不同，本实验的 ARM64 实现重点优化单条消息的分组压缩过程。SM3 的消息扩展和 64 轮压缩均包含较强的数据依赖，若将连续扩展字或连续压缩轮直接映射到不同 NEON lane，会出现同一向量内部前后结果相互依赖的问题。因此，本实验没有简单地将全部计算机械地向量化，而是根据不同阶段的数据并行性，将规则且相互独立的输入处理交给 NEON SIMD 寄存器，将递推消息扩展和压缩关键路径交给 ARM64 通用寄存器。

ARM64 部分依次实现了标准 C 参考版本、NEON intrinsics 混合优化版本和手写汇编版本。NEON intrinsics 版本主要用于验证 SIMD 与通用寄存器协同方案的正确性和初步性能收益；手写汇编版本进一步固定寄存器分配，并通过 16 字消息窗口、轮函数展开、在线派生字计算和访存削减提高整体吞吐率。

3.4.1 NEON 与通用寄存器的混合数据通路

SM3 每个消息分组包含 64 字节，即 16 个 32 bit 消息字。由于 SM3 按大端字节序解释消息字，而鲲鹏 920 运行于小端模式，标准 C 版本需要逐字节组合得到 W_0 至 W_{15} 。在 ARM64 优化版本中，程序使用四个 128 bit NEON 寄存器一次装载完整分组，并使用 rev32 对每个 32 bit lane 内部的四个字节进行翻转。

对应的核心指令如下：

1	ld1	{v0.16b, v1.16b, v2.16b, v3.16b}, [x1]
2		
3	rev32	v0.16b, v0.16b
4	rev32	v1.16b, v1.16b
5	rev32	v2.16b, v2.16b
6	rev32	v3.16b, v3.16b

完成转换后，四个向量寄存器在逻辑上分别保存：

$$\begin{aligned}
 v_0 &= (W_0, W_1, W_2, W_3), \\
 v_1 &= (W_4, W_5, W_6, W_7), \\
 v_2 &= (W_8, W_9, W_{10}, W_{11}), \\
 v_3 &= (W_{12}, W_{13}, W_{14}, W_{15}).
 \end{aligned}$$

NEON 在该阶段能够用较少的装载和字节重排指令完成完整分组的初始解析。对于后续消息扩展，由于连续扩展字存在组内依赖，本实验将 W_{16} 至 W_{67} 的递推计算放

入通用寄存器。压缩状态 A 至 H 、轮常数以及中间变量同样主要保存在 32 bit 通用寄存器中。

因此，ARM64 实现形成了如下混合数据通路：

$$\text{消息分组} \xrightarrow{\text{NEON 装载与字节序转换}} W_0, \dots, W_{15} \xrightarrow{\text{GPR 消息扩展}} W_{16}, \dots, W_{67} \xrightarrow{\text{GPR 压缩}} V^{(i+1)}.$$

这种划分并不追求最大化 SIMD 指令数量，而是将 NEON 用于具有规则并行性的操作，将具有链式依赖的运算保留在通用寄存器中，从而避免频繁的 NEON lane 提取和 SIMD-GPR 数据传递开销。

表 4: ARM64 混合寄存器实现中的任务划分

计算阶段	主要寄存器	代表指令	作用
分组装载	NEON	ld1	一次装载完整的 64 字节消息分组
字节序转换	NEON	rev32	并行完成四个 32 bit 字的大端转换
消息扩展	GPR	eor、ror	使用 16 字滑动窗口递推生成扩展字
压缩状态	GPR	w8 至 w15 等	长期保存 A 至 H
轮常数	GPR	movz、movk、ror	装载并逐轮更新 $T_j \lll j$
压缩轮	GPR	add、eor、and、ror	执行布尔函数、模加和状态更新
派生消息字	GPR	ldr、eor	在压缩轮中按需计算 W'_j

3.4.2 基于通用寄存器滑动窗口的消息扩展

标准 SM3 消息扩展使用长度为 68 的数组保存全部消息字。对于 $16 \leq j \leq 67$ ，扩展公式为：

$$W_j = P_1(W_{j-16} \oplus W_{j-9} \oplus (W_{j-3} \lll 15)) \oplus (W_{j-13} \lll 7) \oplus W_{j-6}.$$

若直接按照数组形式实现，每生成一个新消息字都需要从栈中重新读取 W_{j-16} 、 W_{j-9} 、 W_{j-3} 、 W_{j-13} 和 W_{j-6} ，导致大量重复访存。由于 W_j 仅依赖最近 16 个消息字，本实验在手写汇编版本中使用 16 个通用寄存器构造滑动窗口。

设当前寄存器窗口为：

$$(R_0, R_1, \dots, R_{15}) = (W_{j-16}, W_{j-15}, \dots, W_{j-1}).$$

生成 W_j 后，原来的 W_{j-16} 已不再被后续扩展使用，因此可以直接用 W_j 覆盖对应寄存器。随着扩展过程推进，16 个物理寄存器循环承担不同消息字的逻辑角色，而不需要反复从栈中读取旧消息字。

汇编实现使用宏 EXPAND_REG 描述单个消息字扩展：

```

1 eor    D, D, R7
2 eor    D, D, R13, ror #17
3
4 eor    tmp, D, D, ror #17
5 eor    tmp, tmp, D, ror #9
6 eor    tmp, tmp, R3, ror #25
7 eor    D, tmp, R10

```

其中，32 bit 循环左移 15 位等价于循环右移 17 位，循环左移 23 位等价于循环右移 9 位，循环左移 7 位等价于循环右移 25 位。ARM64 的逻辑指令允许源操作数附带循环移位，因此可以将旋转和异或合并到一条指令中。例如：

```

1 eor    w24, w24, w15, ror #17

```

可以同时完成循环移位和异或，减少独立旋转指令以及临时寄存器使用。

在消息扩展阶段，16 个窗口字长期保存在通用寄存器中，仅在生成新的 W_j 后将结果顺序写入消息字区域，供后续压缩轮访问。与逐字从栈中加载五个依赖项的实现相比，该方法将扩展阶段的主要内存访问缩减为初始 16 个字的装载和新扩展字的必要写入。

3.4.3 压缩轮的寄存器轮换与完全展开

SM3 压缩过程包含 64 轮迭代。若按照标准 C 结构实现，每轮结束后需要执行：

$$D \leftarrow C, \quad C \leftarrow B \lll 9, \quad B \leftarrow A, \quad A \leftarrow TT_1, \\ H \leftarrow G, \quad G \leftarrow F \lll 19, \quad F \leftarrow E, \quad E \leftarrow P_0(TT_2).$$

直接进行上述状态赋值会引入大量寄存器搬移。为减少该开销，本实验使用汇编宏改变逻辑变量与物理寄存器之间的对应关系。连续四轮分别按照如下参数顺序调用：

```

Round(A, B, C, D),
Round(D, A, B, C),
Round(C, D, A, B),
Round(B, C, D, A).

```

经过四轮后，状态字重新回到原来的物理寄存器位置，因此无需在每一轮结束时显式移动 A 至 D 。 E 至 H 采用相同的轮换方式。

由于前 16 轮和后 48 轮采用不同的布尔函数，本实验分别定义：

```

1 .macro ROUND_0_15 A, B, C, D, E, F, G, H
2     ...
3 .endm
4
5 .macro ROUND_16_63 A, B, C, D, E, F, G, H
6     ...
7 .endm

```

前 16 轮使用：

$$FF_j(X, Y, Z) = GG_j(X, Y, Z) = X \oplus Y \oplus Z,$$

后 48 轮使用：

$$FF_j(X, Y, Z) = (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z),$$

$$GG_j(X, Y, Z) = (X \wedge Y) \vee (\neg X \wedge Z).$$

将两类轮函数分离后，压缩过程中不再需要判断轮号是否小于 16。

轮常数通过 `movz` 和 `movk` 装入通用寄存器，并在每轮结束后循环左移一位：

```

1 movz    w17, #0x4519
2 movk    w17, #0x79cc, lsl #16
3 ror     w17, w17, #31

```

最终汇编实现将前 16 轮展开为 4 组四轮宏，将后 48 轮展开为 12 组四轮宏。汇编器展开后形成完整的 64 轮直线代码，从而消除运行时循环计数、条件分支和状态下标计算。该方法会增加代码体积，但能够缩短压缩关键路径，并使寄存器分配和指令顺序更加可控。

3.4.4 在线派生消息字计算与访存削减

标准实现通常预先生成完整的派生消息数组：

$$W'_j = W_j \oplus W_{j+4}, \quad 0 \leq j \leq 63.$$

若保存完整的 $W'[64]$ ，需要额外占用 256 字节栈空间，同时还会产生一次完整写入和压缩阶段的再次读取。为减少中间数据，本实验在最终汇编版本中不再保存独立的

W' 数组，而是在第 j 轮开始时直接装载 W_j 和 W_{j+4} ，并使用一条异或指令得到当前轮需要的 W'_j ：

```
1 ldr    w3, [x1]
2 ldr    w2, [x1, #16]
3 eor    w2, w2, w3
4 add    x1, x1, #4
```

其中， $w3$ 保存 W_j ， $w2$ 最终保存 $W'_j = W_j \oplus W_{j+4}$ 。这样可以删除完整的派生消息字数组，缩小栈帧，并减少数据在 SIMD、内存和通用寄存器之间的往返传递。

在进一步优化的汇编实现中，消息扩展和压缩按照轮次需求进行组织。前若干轮直接使用初始消息字；当压缩轮需要新的 W_{j+4} 时，程序先在 16 字滑动窗口中生成相应扩展字，再立即完成该轮的 W'_j 计算和压缩操作。该方式避免先生成全部扩展字后再执行全部压缩轮，使消息扩展与压缩关键路径更加紧密，并进一步降低中间存储和内存访问开销。

综上，ARM64 优化并不是单纯将标准 C 运算替换为 NEON 指令，而是根据 SM3 内部数据依赖进行执行资源划分：NEON 负责规则且适合并行的消息装载和字节序转换，通用寄存器负责递推消息扩展和强依赖的 64 轮压缩；在此基础上，通过 16 字寄存器窗口、桶形移位操作数、四轮状态轮换、64 轮完全展开和在线派生消息字计算，形成适合鲲鹏 920 平台的 SIMD 与通用寄存器混合优化实现。

4 实验设计与实现

4.1 总体设计

本实验采用分层、分架构的实现方式。首先以标准 C 语言完成 SM3 标量参考实现，提供消息初始化、分段更新、填充和摘要输出等基础功能，并将其作为后续优化版本的正确性基准。在此基础上，分别针对 x86-64 和 ARM64 架构设计优化后端：x86 部分采用 AVX2 和 AVX-512 实现多消息并行，并通过运行时指令集检测选择当前平台支持的最优实现；ARM64 部分采用 NEON intrinsic 和手写汇编实现寄存器级优化。

在工程结构上，公共标量实现与架构相关优化代码相互分离，不同指令集源文件使用各自的编译参数独立编译。该设计既避免了高等级指令集代码影响程序在低等级平台上的正常启动，也便于分别进行正确性测试、反汇编检查和性能比较。实验最终形成 Scalar、AVX2、AVX-512、ARM64 NEON 和 ARM64 Assembly 等多种实现路径。

4.2 标量参考实现

标量参考实现位于 `sm3_c/sm3.c`，其接口声明位于 `sm3_c/sm3.h`。该版本采用 32 bit 无符号整数实现 SM3，按照标准算法顺序完成消息填充、消息扩展和 64 轮压缩，作

为 SIMD 优化版本的正确性参考。

核心接口如下：

```
1 void sm3_init(sm3_ctx *ctx);
2
3 void sm3_update(
4     sm3_ctx *ctx,
5     const uint8_t *data,
6     size_t length
7 );
8
9 void sm3_final(
10     sm3_ctx *ctx,
11     uint8_t digest[32]
12 );
13
14 void sm3_digest(
15     const uint8_t *data,
16     size_t length,
17     uint8_t digest[32]
18 );
```

`sm3_init` 用于设置初始向量并清空上下文状态；`sm3_update` 支持对输入消息进行分段处理，并缓存不足一个 512 bit 分组的数据；`sm3_final` 完成消息填充、长度字段追加和最终摘要输出；`sm3_digest` 则封装上述过程，为单次完整消息计算提供简化接口。

标量版本能够处理任意长度输入，并负责消息缓存、分组处理和填充。当批量消息数量不足 SIMD 向量宽度时，运行时分发模块也可以使用该版本完成尾部消息处理。

4.3 AVX2 八路并行实现

AVX2 八路并行实现位于 `sm3_x86/sm3_avx2.c`。该版本使用 `__m256i` 表示包含 8 个 32 bit lane 的向量数据，使 8 条独立消息在相同轮次下同时执行消息扩展和压缩运算。

基本向量操作定义如下：

```
1 typedef __m256i v8u32;
2
3 #define V8_ADD(a, b) \
4     _mm256_add_epi32((a), (b))
5
```

```
6 #define V8_XOR(a, b) \  
7     _mm256_xor_si256((a), (b))  
8  
9 #define V8_AND(a, b) \  
10    _mm256_and_si256((a), (b))  
11  
12 #define V8_OR(a, b) \  
13    _mm256_or_si256((a), (b))  
14  
15 #define V8_SET1(x) \  
16    _mm256_set1_epi32((int)(x))  
17  
18 #define V8_ROTL32(x, n) \  
19    V8_OR( \  
20        _mm256_slli_epi32((x), (n)), \  
21        _mm256_srli_epi32((x), 32 - (n)) \  
22    )
```

其中, $V8_ADD$ 对 8 个 32 bit lane 分别进行模 2^{32} 加法; $V8_XOR$ 、 $V8_AND$ 和 $V8_OR$ 分别实现向量异或、与和或运算。由于 AVX2 没有直接提供任意 32 bit 循环左移 intrinsic, 程序通过左移、右移和按位或组合实现 $V8_ROTL32$ 。

向量化置换函数 P_0 和 P_1 实现为:

```
1 static inline v8u32 v8_p0(v8u32 x)  
2 {  
3     return V8_XOR(  
4         V8_XOR(x, V8_ROTL32(x, 9)),  
5         V8_ROTL32(x, 17)  
6     );  
7 }  
8  
9 static inline v8u32 v8_p1(v8u32 x)  
10 {  
11     return V8_XOR(  
12         V8_XOR(x, V8_ROTL32(x, 15)),  
13         V8_ROTL32(x, 23)  
14     );  
15 }
```

每个 $v8u32$ 变量保存 8 条消息在相同位置的状态字, 因此上述函数能够在一次调用中并行计算 8 组 P_0 或 P_1 。

消息扩展阶段一次生成 8 条消息的同一位置扩展字：

```
1 for (int j = 16; j < 68; ++j) {
2     v8u32 t = V8_XOR(
3         V8_XOR(W[j - 16], W[j - 9]),
4         V8_R0TL32(W[j - 3], 15)
5     );
6
7     W[j] = V8_XOR(
8         V8_XOR(
9             v8_p1(t),
10            V8_R0TL32(W[j - 13], 7)
11        ),
12        W[j - 6]
13    );
14 }
15
16 for (int j = 0; j < 64; ++j) {
17     W1[j] = V8_XOR(W[j], W[j + 4]);
18 }
```

数组 $W[j]$ 的每个元素均为一个 YMM 向量，其 8 个 lane 分别保存 8 条消息的第 j 个扩展字。因此，消息扩展公式中的异或、移位和置换操作都可以在 8 条消息之间并行执行。

输入数据从按消息组织的 AoS 逻辑布局转换为按状态字聚合的 SoA 向量布局，将 8 条消息相同位置的 32 bit 数据装入同一个 YMM 寄存器。对于长度不同的批量消息，程序维护活动 lane 掩码；当某个 lane 已经完成全部分组时，通过

```
1 _mm256_blendv_epi8(
2     old_state,
3     new_state,
4     active_mask
5 )
```

保留该 lane 原有状态，只提交仍处于活动状态的 lane 的计算结果。

4.4 AVX-512 十六路并行实现

AVX-512 十六路并行实现位于 `sm3_x86/sm3_avx512.c`。该版本沿用 AVX2 的 Multi-Buffer 数据布局，使用 `__m512i` 将并行宽度扩展为 16 个 32 bit lane。

主要基本操作定义如下：

```
1 typedef __m512i v16u32;
2
3 #define V16_ADD(a, b) \
4     _mm512_add_epi32((a), (b))
5
6 #define V16_XOR(a, b) \
7     _mm512_xor_si512((a), (b))
8
9 #define V16_SET1(x) \
10     _mm512_set1_epi32((int)(x))
11
12 #define V16_ROTL32(x, n) \
13     _mm512_rol_epi32((x), (n))
```

与 AVX2 相比, AVX-512 可以直接通过 `_mm512_rol_epi32` 完成 32 bit lane 内的循环左移, 减少由两次移位和一次按位或组成的指令序列。

AVX-512 版本使用 `__mmask16` 记录 16 个 lane 的活动状态, 并通过掩码移动指令选择性更新链值:

```
1 state_a = _mm512_mask_mov_epi32(
2     state_a,
3     active_mask,
4     next_a
5 );
```

当前实验主机不支持 AVX-512F, 因此只在本地完成了该版本的源码编译、静态反汇编和运行时安全跳过验证。

4.5 运行时指令集分发

运行时分发逻辑位于 `sm3_x86/sm3_dispatch.c`。为避免在不支持 AVX-512 的处理器上误执行相关指令, 程序不会在编译时固定使用某一个后端, 而是在运行时检测 CPU 和操作系统实际开放的指令集能力。

核心检测逻辑如下:

```
1 __builtin_cpu_init();
2
3 if (__builtin_cpu_supports("avx512f")) {
4     /* 使用 AVX-512 十六路后端 */
5 } else if (__builtin_cpu_supports("avx2")) {
6     /* 使用 AVX2 八路后端 */
```

```
7 } else {  
8     /* 回退到标量后端 */  
9 }
```

批量消息调度优先级为：

AVX-512 $\rightarrow$ AVX2 $\rightarrow$ Scalar.

当处理器支持 AVX-512F 且当前批次中至少包含 16 条消息时，程序优先调用 AVX-512 十六路后端；否则，在支持 AVX2 且批次中至少包含 8 条消息时，调用 AVX2 八路后端；剩余不足向量宽度的尾部消息由标量实现处理。

在当前实验主机上，运行时检测结果为：

```
CPU AVX2:      yes  
CPU AVX-512: no  
Auto backend: AVX2 (8-way)
```

该结果表明程序能够正确识别处理器能力，并在 Intel Core i7-13700H 平台上自动选择 AVX2 八路实现。

4.6 ARM64 实现

ARM64 优化实现位于 `sm3_arm/sm3_arm64.S`。该版本使用 AArch64 手写汇编实现单个 512 bit 消息分组的 SM3 压缩函数，并通过 NEON SIMD 寄存器和通用寄存器的协同，完成消息装载、字节序转换、消息扩展和 64 轮压缩。

公共 C 层仍负责消息初始化、分段更新、填充和摘要输出。ARM64 汇编后端只实现统一的单分组压缩接口：

```
1 void sm3_compress_arm64_asm(  
2     uint32_t state[SM3_STATE_WORDS],  
3     const uint8_t block[SM3_BLOCK_SIZE]  
4 );
```

其中，`state` 指向 8 个 32 bit 链值，`block` 指向一个未经字节序转换的 64 字节消息分组。公共实现通过编译期宏将压缩函数映射到 ARM64 汇编后端：

```
1 #if defined(SM3_USE_ARM64_ASM)  
2  
3 #if !defined(__aarch64__)  
4 #error "SM3_USE_ARM64_ASM requires an AArch64 target"  
5 #endif  
6  
7 #define SM3_COMPRESS_BLOCK sm3_compress_arm64_asm
```

```
8
9 #else
10
11 #define SM3_COMPRESS_BLOCK sm3_compress_ref
12
13 #endif
```

因此,参考 C 版本与 ARM64 汇编版本复用相同的 `sm3_init`、`sm3_update`、`sm3_final` 和 `sm3_digest`, 两者的区别仅位于单分组压缩函数内部。

4.6.1 SIMD 装载与寄存器分配

SM3 每个消息分组包含 64 字节, 即 16 个 32 bit 消息字。鲲鹏 920 运行于小端模式, 而 SM3 以大端方式解释输入消息字。汇编实现首先使用四个 128 bit NEON 寄存器一次装载完整分组, 然后使用 `rev32` 同时反转每个 32 bit lane 内部的字节:

```
1 ld1      {v0.16b, v1.16b, v2.16b, v3.16b}, [x1]
2
3 rev32     v0.16b, v0.16b
4 rev32     v1.16b, v1.16b
5 rev32     v2.16b, v2.16b
6 rev32     v3.16b, v3.16b
```

完成字节序转换后, 四个向量寄存器在逻辑上分别保存 W_0 至 W_{15} 。程序将转换结果一次性写入 64 字节的栈上临时区域, 再通过成对装载指令转入通用寄存器。该临时区域仅作为 NEON 到 GPR 的一次性数据桥接, 后续消息扩展和压缩过程中不再频繁进行 SIMD lane 提取。

最终汇编版本同时将 16 字消息窗口和 8 个压缩状态字保存在通用寄存器中。具体分配如表 5 所示。

表 5: ARM64 汇编实现的寄存器分配

用途	寄存器	说明
初始消息装载	v0 至 v3	一次装载 64 字节，并行完成 16 个消息字的字节序转换
消息扩展窗口前半部分	w0 至 w7	保存 16 字循环窗口中的前 8 个消息字
消息扩展窗口后半部分	w19 至 w26	保存 16 字循环窗口中的后 8 个消息字
压缩状态	w8 至 w15	分别保存状态字 A 、 B 、 C 、 D 、 E 、 F 、 G 、 H
轮常数	w17	保存当前轮使用的 $T_j \lll j$
临时变量	w16、w27 至 w30	用于消息扩展、布尔函数、 SS_1 、 SS_2 和中间加法
地址与栈管理	x0、x1、x16、sp	保存状态地址、消息地址、临时区域地址和栈指针
被调用者保存寄存器	x19 至 x30	按照 AArch64 调用约定在函数入口保存、退出前恢复

实现中没有使用 x18，以避免与部分 AArch64 系统中的平台保留寄存器发生冲突。

4.6.2 基于通用寄存器的消息扩展窗口

SM3 消息扩展公式为：

$$W_j = P_1(W_{j-16} \oplus W_{j-9} \oplus (W_{j-3} \lll 15)) \oplus (W_{j-13} \lll 7) \oplus W_{j-6}.$$

由于 W_j 仅依赖最近 16 个消息字，汇编版本不保存完整的 $W[68]$ 数组，而是使用 16 个通用寄存器构造循环窗口。生成 W_j 后， W_{j-16} 已不再被后续扩展使用，因此可以直接使用新结果覆盖原寄存器。

单个消息扩展字由 EXPAND_REG 宏实现：

```

1 .macro EXPAND_REG D, R7, R13, R3, R10
2     eor    \D, \D, \R7
3     eor    \D, \D, \R13, ror #17
4
5     eor    w16, \D, \D, ror #17

```

```

6      eor      w16, w16, \D, ror #9
7      eor      w16, w16, \R3, ror #25
8      eor      \D, w16, \R10
9      .endm

```

宏参数 D 初始保存 W_{j-16} ，计算完成后被覆盖为 W_j 。其余参数依次对应 W_{j-9} 、 W_{j-3} 、 W_{j-13} 和 W_{j-6} 。

32 bit 循环左移 15、23 和 7 位分别等价于循环右移 17、9 和 25 位。AArch64 逻辑指令允许第二个源操作数附带循环移位，因此：

```

1      eor      D, D, R13, ror #17

```

能够在一条指令中同时完成循环移位和异或，避免额外的旋转指令及临时寄存器。

4.6.3 消息扩展与压缩的交错执行

汇编版本采用 on-the-fly 调度，不先生成完整的 W_0 至 W_{67} ，也不保存独立的 W'_0 至 W'_{63} 数组。

压缩轮 0 至 11 所需的 W_j 和 W_{j+4} 均位于初始的 W_0 至 W_{15} 中，因此可以直接执行。对应代码按照四轮状态轮换方式展开：

```

1      STEP_0_15 w0, w4,  w8,  w9,  w10, w11,
2                      w12, w13, w14, w15
3
4      STEP_0_15 w1, w5,  w11, w8,  w9,  w10,
5                      w15, w12, w13, w14
6
7      STEP_0_15 w2, w6,  w10, w11, w8,  w9,
8                      w14, w15, w12, w13
9
10     STEP_0_15 w3, w7,  w9,  w10, w11, w8,
11                      w13, w14, w15, w12

```

从第 12 轮开始，第 j 轮需要新的 W_{j+4} 。程序先通过 EXPAND_REG 生成该扩展字，随后立即执行对应压缩轮。例如：

```

1      EXPAND_REG w0, w7, w24, w3, w21
2
3      STEP_0_15 w23, w0,
4                  w8,  w9,  w10, w11,
5                  w12, w13, w14, w15

```

其中，第一条宏调用生成新的 W_{16} ，第二条宏调用立即使用 W_{12} 和 W_{16} 完成第 12 轮压缩。

剩余轮次通过：

```
1 CYCLE_12_27
2 CYCLE_16
3 CYCLE_16
4 CYCLE_4
```

依次完成。每个循环宏在汇编期展开，不包含运行时循环分支。因此，从第 12 轮至第 63 轮均遵循：

生成 $W_{j+4} \rightarrow$ 计算 $W'_j \rightarrow$ 执行第 j 轮。

派生消息字在每轮内部直接计算：

```
1 eor    w27, W, W4
```

其中， W 和 $W4$ 分别表示 W_j 和 W_{j+4} ，因此：

$$w27 = W_j \oplus W_{j+4} = W'_j.$$

这种实现不需要完整的 $W[68]$ 和 $W'[64]$ 数组，避免了消息扩展结果的集中写回以及压缩阶段的再次读取。

4.6.4 压缩轮宏与指令调度

由于 SM3 前 16 轮和后 48 轮采用不同的布尔函数，汇编版本分别定义 `ROUND_0_15` 和 `ROUND_16_63`。

前 16 轮的主要逻辑为：

$$FF_j(A, B, C) = A \oplus B \oplus C, \quad GG_j(E, F, G) = E \oplus F \oplus G.$$

其汇编实现片段如下：

```
1 eor    w16, A, B
2 eor    w16, w16, C
3 add    D, D, w16
4
5 eor    w16, E, F
6 eor    w16, w16, G
7 add    H, H, w16
```

后 48 轮中的 FF_j 使用：

$$FF_j(A, B, C) = (A \wedge (B \oplus C)) \oplus (B \wedge C),$$

对应代码为：

```
1 eor    w16, B, C
2 and    w16, A, w16
3 and    w28, B, C
4 eor    w16, w16, w28
```

GG_j 使用等价形式：

$$GG_j(E, F, G) = G \oplus (E \wedge (F \oplus G)),$$

对应代码为：

```
1 eor    w16, F, G
2 and    w16, E, w16
3 eor    w16, w16, G
```

压缩轮中的 D 和 H 更新形成两条相对独立的加法链。汇编实现将两条链交错排列：

```
1 add    H, H, w16
2 add    D, D, w30
3 add    H, H, w29
4 add    D, D, w27
5 add    H, H, W
```

这样可以降低单条长依赖链对执行吞吐率的限制，使整数执行单元能够在两组独立运算之间交替推进。

轮常数保存在 $w17$ 中，初始常数通过 `movz` 和 `movk` 构造：

```
1 movz    w17, #0x4519
2 movk    w17, #0x79cc, lsl #16
```

每轮结束后执行：

```
1 ror     w17, w17, #31
```

该指令等价于 32 bit 循环左移一位，从而得到下一轮使用的 $T_j \lll j$ 。

4.6.5 栈帧与状态回写

汇编函数使用 176 字节栈帧，其布局为：

表 6: ARM64 汇编函数栈帧布局

栈偏移	大小	用途
0–95	96 字节	保存被调用者保存寄存器 x19 至 x30
96–103	8 字节	保存输入的 state 指针
104–111	8 字节	对齐和保留空间
112–175	64 字节	初始消息从 NEON 寄存器转入通用寄存器的一次性中转区域

栈帧大小为 176 字节，是 16 的整数倍，满足 AArch64 调用约定对栈指针对齐的要求。函数入口保存 x19 至 x30，函数退出前恢复这些寄存器。

完成 64 轮压缩后，程序重新读取原始链值，与寄存器中的最终状态逐字异或并写回：

```
1 ldp    w16, w17, [x0, #0]
2 eor    w8, w8, w16
3 eor    w9, w9, w17
4 stp    w8, w9, [x0, #0]
```

其余状态字采用相同方式处理，最终完成：

$$V^{(i+1)} = V^{(i)} \oplus (A_{64}, B_{64}, C_{64}, D_{64}, E_{64}, F_{64}, G_{64}, H_{64}).$$

综上，ARM64 最终实现以 NEON 完成规则的消息装载和字节序转换，并通过一次性栈中转将数据交给通用寄存器；随后使用同时驻留于 GPR 的 16 字消息窗口和 8 个压缩状态，交错完成消息扩展与 64 轮压缩。该实现结合了 NEON SIMD 数据通路、ARM64 桶形移位操作数、循环窗口、在线派生字计算、汇编期轮展开和指令级并行调度，形成了完整的 SIMD 寄存器与通用寄存器混合优化后端。

5 实验结果与分析

5.1 x86 实验结果与分析

5.1.1 功能正确性验证

运行 x86 正确性测试程序，对 AVX2 八路并行实现、运行时指令集检测和后端分发逻辑进行验证。在 sm3_x86 目录下执行：

```
make test
```

程序输出结果如下：

```
• (base) whisper@whisper:~/sdu-cybersecurity-innovation-practice-2026-team41/7.23-作业4-付勇-SM2软件实现与优化/sm3_x86$  
make test  
./sm3_x86_test  
CPU AVX2: yes  
CPU AVX-512: no  
Auto backend: AVX2 (8-way)  
[PASS] AVX2 edge and random differential tests  
[SKIP] AVX-512 tests: unsupported CPU/OS  
[PASS] runtime dispatch test  
All SM3 x86 tests passed.
```

测试结果表明，AVX2 实现能够正确处理常规长度消息、消息填充边界以及随机生成的不同长度消息。各个 SIMD lane 产生的摘要均与标量参考实现一致，说明多消息并行、消息扩展、活动 lane 管理和压缩函数实现正确。

运行时检测结果显示，当前 Intel Core i7-13700H 支持 AVX2，但不支持 AVX-512F，因此程序自动选择 AVX2 八路后端，并安全跳过 AVX-512 动态测试。运行过程中未出现非法指令异常，说明指令集检测和运行时分发逻辑工作正常。

为了避免标量实现和 SIMD 实现同时存在相同错误，实验进一步使用 OpenSSL 提供的 SM3 实现作为独立参考。执行命令：

```
make verify-openssl
```

输出结果为：

```
[PASS] 149 files matched OpenSSL SM3;  
backend=Auto backend: AVX2 (8-way)
```

差分验证程序生成不同长度和不同内容的测试文件，分别调用本实验实现和 OpenSSL 计算 SM3 摘要，并逐字节比较两者的输出。149 组测试文件的摘要全部一致，进一步证明了标量实现、AVX2 八路实现和运行时分发接口的正确性。

5.1.2 内存安全性检查

在功能正确性验证的基础上，实验使用 AddressSanitizer 和 UndefinedBehaviorSanitizer 检查程序中可能存在的内存错误和未定义行为。

执行：

```
make sanitize
```

该目标使用以下主要参数重新编译测试程序：

```
-O1 -g  
-fsanitize=address,undefined  
-fno-omit-frame-pointer
```

Sanitizer 版本运行后仍然输出：

```
[PASS] AVX2 edge and random differential tests
[SKIP] AVX-512 tests: unsupported CPU/OS
[PASS] runtime dispatch test
All SM3 x86 tests passed.
```

测试过程中未出现 AddressSanitizer 或 UndefinedBehaviorSanitizer 报错，说明当前测试范围内未发现堆栈越界、非法内存访问、释放后使用、非法移位或其他 C 语言未定义行为。

5.1.3 反汇编验证

为了确认 intrinsic 源码被编译为真实的 SIMD 指令，而不是由编译器退化为普通标量运算，实验对 AVX2 和 AVX-512 目标文件进行反汇编检查。

执行：

```
make asm-check
```

在 AVX2 目标文件中可以观察到：

```
vinseriti128 ymm0,ymm0,xmm1,0x1
vmovdqa YMMWORD PTR [rdi+rax*8],ymm0
vmovdqa ymm3,YMMWORD PTR [rsp+0xb20]
vpsrld ymm2,ymm3,0x11
```

其中，ymm0、ymm2 和 ymm3 为 256 bit YMM 寄存器，vmovdqa 用于向量数据装载和保存，vpsrld 用于对 8 个 32 bit lane 同时执行逻辑右移。这些指令说明 AVX2 版本确实生成了八路并行机器代码。

AVX-512 目标文件中可以观察到：

```
vmovdqa32 zmm3,ZMMWORD PTR [rsp+0x40]
vmovdqa64 zmm4,ZMMWORD PTR [rsp+0x15c0]
vprold zmm4,zmm4,0xf
```

其中，zmm 为 512 bit 向量寄存器，vprold 可以直接对 16 个 32 bit lane 执行循环左移。这表明 AVX-512 十六路实现已经被正确编译为 ZMM 向量指令。

```

(base) whisper@whisper:~/sdu-cybersecurity-innovation-practice-2026-team41/7.23-作业4-付勇-SM2软件实现与优化/sm3_x86$
make asm-check
objdump -d -Mintel build/sm3_avx2.o > build/sm3_avx2.asm
objdump -d -Mintel build/sm3_avx512.o > build/sm3_avx512.asm
AVX2 YMM instructions:
b5: c4 e3 7d 38 c1 01 vinserti128 ymm0,ymm0,xmm1,0x1
bb: c5 fd 7f 04 c7 vmovdqa YMMWORD PTR [rdi+rax*8],ymm0
ca: c5 fd 6f 9c 24 20 0b vmovdqa ymm3,YMMWORD PTR [rsp+0xb20]
d8: c5 fd 6f 84 24 40 0b vmovdqa ymm0,YMMWORD PTR [rsp+0xb40]
e9: c5 7d 6f 94 24 60 0b vmovdqa ymm10,YMMWORD PTR [rsp+0xb60]
f2: c5 7d 6f 8c 24 60 0a vmovdqa ymm9,YMMWORD PTR [rsp+0xa60]
fb: c5 7d 6f 84 24 80 0a vmovdqa ymm8,YMMWORD PTR [rsp+0xa80]
104: c5 fd 6f bc 24 a0 0a vmovdqa ymm7,YMMWORD PTR [rsp+0xaa0]
10d: c5 fd 6f b4 24 80 09 vmovdqa ymm6,YMMWORD PTR [rsp+0x980]
116: c5 fd 6f ac 24 a0 09 vmovdqa ymm5,YMMWORD PTR [rsp+0x9a0]
11f: c5 fd 6f 8c 24 c0 09 vmovdqa ymm1,YMMWORD PTR [rsp+0x9c0]
128: c5 ed 72 d3 11 vpsrld ymm2,ymm3,0x11
AVX-512 ZMM/mask instructions:
1a2: 62 f1 7d 48 6f 5c 24 vmovdqa32 zmm3,ZMMWORD PTR [rsp+0x40]
1aa: 62 f1 7d 48 7f 1c 11 vmovdqa32 ZMMWORD PTR [rcx+rdx*1],zmm3
1bb: 62 f1 fd 48 6f 64 24 vmovdqa64 zmm4,ZMMWORD PTR [rsp+0x15c0]
1c8: 62 f1 fd 48 6f 5c 24 vmovdqa64 zmm3,ZMMWORD PTR [rsp+0x1600]
1d5: 62 f1 fd 48 6f 54 24 vmovdqa64 zmm2,ZMMWORD PTR [rsp+0x1640]
1dd: 62 71 fd 48 6f 54 24 vmovdqa64 zmm10,ZMMWORD PTR [rsp+0x1440]
1ed: 62 71 fd 48 6f 4c 24 vmovdqa64 zmm9,ZMMWORD PTR [rsp+0x1480]
1f5: 62 71 fd 48 6f 44 24 vmovdqa64 zmm8,ZMMWORD PTR [rsp+0x14c0]
1fd: 62 f1 fd 48 6f 7c 24 vmovdqa64 zmm7,ZMMWORD PTR [rsp+0x1280]
205: 62 f1 fd 48 6f 74 24 vmovdqa64 zmm6,ZMMWORD PTR [rsp+0x12c0]
20d: 62 f1 fd 48 6f 6c 24 vmovdqa64 zmm5,ZMMWORD PTR [rsp+0x1300]
215: 62 f1 5d 48 72 cc 0f vprold zmm4,zmm4,0xf

```

除 SIMD 寄存器外，AVX2 函数中还使用了大量通用寄存器：

```

mov r13,QWORD PTR [rsi]
mov r12,QWORD PTR [rsi+0x8]
mov rbx,QWORD PTR [rsi+0x10]
mov rax,rsi
mov r8,rdi
mov r11,QWORD PTR [rax+0x20]
mov r10,QWORD PTR [rax+0x28]

```

其中，rsi、rax、rbx、r8、r10 和 r11 主要用于保存消息指针、数组基址、输出地址和偏移信息。因此，最终机器代码采用了通用寄存器负责地址与控制调度、SIMD 寄存器负责多路数据运算的混合实现结构。

```

(base) whisper@whisper:~/sdu-cybersecurity-innovation-practice-2026-team41/7.23-作业4-付勇-SM2软件实现与优化/sm3_x86$
grep -E \
  '\b(rax|rbx|rcx|rdx|rsi|rdi|r8|r9|r10|r11)\b' \
  build/sm3_avx2.asm \
  | head -n 20
8: 53 push rbx
28: 4c 8b 2e mov r13,QWORD PTR [rsi]
2b: 4c 8b 66 08 mov r12,QWORD PTR [rsi+0x8]
2f: 48 8b 5e 10 mov rbx,QWORD PTR [rsi+0x10]
33: 48 89 f0 mov rax,rsi
36: 48 8b 76 18 mov rsi,QWORD PTR [rsi+0x18]
3a: 49 89 f8 mov r8,rdi
40: 4c 8b 58 20 mov r11,QWORD PTR [rax+0x20]
44: 4c 8b 50 28 mov r10,QWORD PTR [rax+0x28]
48: 48 8d bc 24 80 09 00 lea rdi,[rsp+0x980]
50: 64 48 8b 14 25 28 00 mov rdx,QWORD PTR fs:0x28
59: 48 89 94 24 18 12 00 mov QWORD PTR [rsp+0x1218],rdx
63: 48 8b 48 30 mov rcx,QWORD PTR [rax+0x30]
67: 48 8b 50 38 mov rdx,QWORD PTR [rax+0x38]
6d: 0f 1f 00 nop DWORD PTR [rax]
70: c5 f9 6e 24 03 vmovd xmm4,DWORD PTR [rbx+rax*1]
75: c4 e3 59 22 0c 06 01 vpsrld xmm1,xmm4,DWORD PTR [rsi+rax*1],0x1
7c: c4 c1 79 6e 64 05 00 vmovd xmm4,DWORD PTR [r13+rax*1+0x0]
83: c4 c3 59 22 04 04 01 vpsrld xmm0,xmm4,DWORD PTR [r12+rax*1],0x1
8a: c5 f9 6e 24 01 vmovd xmm4,DWORD PTR [rcx+rax*1]

```

5.1.4 性能测试方法

性能测试程序将每个测试批次固定为 16 条消息，使不同实现处理的输入规模保持一致。标量版本依次处理 16 条消息，AVX2 版本调用两次八路并行函数。

实验选择以下六种消息长度：

64, 256, 1024, 4096, 16384, 1048576 byte.

这些测试点分别覆盖短消息、多个分组组成的中等长度消息以及长消息场景。测试时使用：

```
taskset -c 2 ./sm3_x86_bench
```

将进程固定到逻辑 CPU 2，减少进程在不同逻辑处理器之间迁移所造成的调度和缓存波动。

吞吐量按照下式计算：

$$\text{Throughput} = \frac{\text{Processed Bytes}}{\text{Elapsed Time} \times 1024^2} \quad \text{MiB/s.}$$

单位字节处理时间为：

$$\text{ns/B} = \frac{\text{Elapsed Time} \times 10^9}{\text{Processed Bytes}}.$$

AVX2 相对于标量实现的加速比为：

$$\text{Speedup} = \frac{\text{AVX2 Throughput}}{\text{Scalar Throughput}}.$$

测试前保持编译参数、运行环境、消息批量大小和 CPU 亲和性一致，从而保证标量版本与 AVX2 版本之间的对比具有可比性。

5.1.5 性能测试结果

固定逻辑 CPU 核后得到的一组代表性性能结果如表 7 所示。

表 7: x86 标量实现与 AVX2 八路并行实现性能对比

消息长度	Scalar 吞吐率 (MiB/s)	Scalar 时间 (ns/B)	AVX2 吞吐率 (MiB/s)	AVX2 时间 (ns/B)	加速比
64 B	97.77	9.754	621.14	1.535	6.35×
256 B	156.80	6.082	1021.60	0.934	6.52×
1024 B	188.23	5.067	1215.36	0.785	6.46×
4096 B	195.74	4.872	1266.40	0.753	6.47×
16384 B	197.54	4.828	1246.71	0.765	6.31×
1048576 B	196.12	4.863	1233.94	0.773	6.29×

从表中可以看出，AVX2 八路实现对所有测试长度均取得了显著加速。对于 64 B 短消息，AVX2 吞吐率达到 621.14 MiB/s，相对于标量版本提升约 6.35 $\times$ 。这说明即使在填充和调度开销占比较高的短消息场景中，多消息并行仍然能够获得明显收益。

随着消息长度增加，标量版本的吞吐率逐渐稳定在约 196 MiB/s，AVX2 版本则稳定在约 1.2 GiB/s。对于 256 B 至 1 MiB 消息，AVX2 的加速比总体保持在 6.29 $\times$ 至 6.52 $\times$ 之间，说明输入重排和批次调度开销已被较长消息中的压缩函数计算所摊薄。

5.1.6 多轮重复测试

为了降低 WSL2 调度、处理器动态频率和主机后台负载等因素造成的偶然波动，实验对每种消息长度分别执行 10 轮独立测试，并使用中位数作为稳定性能指标。

测试命令如下：

```
for size in 64 256 1024 4096 16384 1048576
do
    for round in $(seq 1 10)
    do
        taskset -c 2 \
            ./sm3_x86_bench "$size" 256
    done > "results/bench-${size}-10runs.txt"
done
```

以 1024 B 消息为例，10 轮结果的中位数如表 8 所示。

表 8: 1024 B 消息十轮重复测试中位数

实现	吞吐率中位数 (MiB/s)	单位字节时间中位数 (ns/B)	加速比中位数
Scalar	188.39	5.063	1.00×
AVX2	1250.62	0.763	6.66×

在 10 轮测试中, 1024 B 消息的结果范围为:

- Scalar 吞吐率为 186.15 至 193.09 MiB/s;
- AVX2 吞吐率为 1159.89 至 1311.15 MiB/s;
- AVX2 加速比为 6.18× 至 6.95×。

标量版本的性能波动相对较小, 而 AVX2 结果存在一定变化。该波动可能与 WSL2 调度、Intel 混合架构中的逻辑处理器映射、动态睿频和 Windows 主机后台任务有关。

5.1.7 加速效果分析

AVX2 寄存器包含 8 个 32 bit lane, 因此从并行宽度上看, 八路实现的理想加速上限为 8×。实际测试中, 不同消息长度下的加速比约为 6.3× 至 6.7×, 低于理论并行宽度, 但符合软件 SIMD 实现的实际情况。造成该差异的主要因素包括:

- **输入重排开销。**原始消息按照消息分别组织, 而 SIMD 计算需要将不同消息中相同位置的 32 bit 字聚合到同一个向量寄存器中, AoS 到 SoA 的转换会产生额外的装载和重排操作。
- **消息填充开销。**每条消息的长度可能不同, 需要分别构造末尾填充分组并追加 64 bit 长度字段, 该过程不能完全使用统一的向量操作完成。
- **标量控制开销。**消息地址、分组数、循环索引和批次状态仍由通用寄存器和标量控制逻辑维护, 无法全部转换为 SIMD 运算。
- **活动 lane 管理开销。**当同一批次中的消息长度不同时, 部分 lane 会提前完成, 程序需要构造活动掩码并使用 blend 指令保留已完成 lane 的状态。
- **尾部消息处理开销。**当剩余消息数量不足 8 条时, 程序需要回退到标量实现, 因而不能始终保持所有 SIMD lane 处于有效工作状态。
- **轮间数据依赖。**每条消息内部的 64 轮压缩仍然具有严格的状态依赖关系, Multi-Buffer 方法只能并行不同消息, 不能消除单个 lane 内部的关键依赖链。

- **处理器执行资源限制。** AVX2 指令仍然受到执行端口数量、寄存器压力、指令调度和动态频率变化的影响，因此向量宽度不能直接等价为实际加速比。

总体而言，1024 B 消息在十轮测试中取得了 $6.66\times$ 的中位加速比，已经实现了较高的 SIMD lane 利用率。长消息吞吐率稳定在约 1.2 GiB/s，说明 Multi-Buffer 数据布局和 AVX2 八路并行设计能够有效提高 SM3 批量消息处理性能。

5.2 ARM64 实验结果与分析

5.2.1 功能正确性验证

ARM64 部分分别对 Reference C、ARM64 NEON 和 ARM64 Assembly 三种实现进行功能正确性验证。三种实现复用相同的公开接口、消息填充逻辑和测试程序，仅替换单个 512 bit 消息分组的压缩后端。

分别构建并运行三种实现后，测试程序均输出：

```
Tests run:      24
Tests passed: 24
Tests failed: 0
All SM3 tests passed.
```

测试内容包括空消息、字符串 abc、字符串 abcd 重复 16 次和一百万个字符 a 等标准测试向量。同时，程序测试了 0、1、55、56、57、63、64、65、119、120、121、127、128、129 和 1024 字节等消息长度，覆盖 SM3 填充过程中的主要边界情况。

流式接口测试分别采用单字节分块、小素数长度分块、接近 64 字节边界的分块和不规则长度分块，并将多次调用 sm3_update 得到的摘要与一次性调用 sm3_digest 的结果进行比较。三种实现的 24 项测试全部通过，说明 Reference C、ARM64 NEON 和 ARM64 Assembly 后端均能够正确完成消息分组、填充、消息扩展、压缩和摘要输出。

```
===== Reference C =====
Tests run:      24
Tests passed: 24
Tests failed: 0
All SM3 tests passed.

===== ARM64 NEON =====
Tests run:      24
Tests passed: 24
Tests failed: 0
All SM3 tests passed.

===== ARM64 Assembly =====
Tests run:      24
Tests passed: 24
Tests failed: 0
All SM3 tests passed.
wyxin@xfusion:~/sm3/sm3-project/sm3_arm$
```

图 3: ARM64 三种实现的功能正确性测试结果

5.2.2 反汇编验证

为了确认最终 ARM64 汇编后端确实使用了 NEON SIMD 寄存器和通用寄存器，实验对汇编目标文件进行反汇编检查。

在函数入口处可以观察到：

```
ld1      {v0.16b-v3.16b}, [x1]

rev32    v0.16b, v0.16b
rev32    v1.16b, v1.16b
rev32    v2.16b, v2.16b
rev32    v3.16b, v3.16b
```

其中，v0 至 v3 为 128 bit NEON 向量寄存器。ld1 一次装载完整的 64 字节消息分组，随后四条 rev32 指令分别对每个 32 bit lane 内部的字节顺序进行转换，从而得到符合 SM3 大端定义的 W_0 至 W_{15} 。

在消息扩展和压缩部分可以观察到带循环移位操作数的通用寄存器指令：

```
eor    w16, w15, w15, ror #23
eor    w15, w16, w15, ror #15
eor    w16, w14, w14, ror #23
eor    w14, w16, w14, ror #15
```

这些指令将循环移位与异或合并执行，用于实现 SM3 中的置换函数和消息扩展运算。反汇编中还出现了：

```
and    w16, w8, w16
and    w28, w9, w10

add    w29, w28, w12
add    w29, w29, w17
add    w11, w11, w16
add    w15, w15, w30
```

其中，and 指令用于实现后 48 轮的布尔函数，add 指令用于完成模 2^{32} 加法和压缩状态更新。

反汇编结果表明，最终实现使用 v0 至 v3 完成 SIMD 消息装载和字节序转换，并使用 w0 至 w30 等通用寄存器完成消息扩展、状态保存、布尔函数和 64 轮压缩。因此，最终机器代码确实采用了 NEON SIMD 数据通路与 GPR 压缩数据通路相结合的混合寄存器结构。

```

===== NEON load =====
20:      4c402020      ld1      {v0.16b-v3.16b}, [x1]
===== NEON byte reversal =====
24:      6e200800      rev32    v0.16b, v0.16b
28:      6e200821      rev32    v1.16b, v1.16b
2c:      6e200842      rev32    v2.16b, v2.16b
30:      6e200863      rev32    v3.16b, v3.16b
===== GPR rotate-and-XOR =====
bc:      4acf5df0      eor      w16, w15, w15, ror #23
c0:      4acf3e0f      eor      w15, w16, w15, ror #15
110:     4ace5dd0      eor      w16, w14, w14, ror #23
114:     4ace3e0e      eor      w14, w16, w14, ror #15
164:     4acd5db0      eor      w16, w13, w13, ror #23
168:     4acd3e0d      eor      w13, w16, w13, ror #15
===== GPR Boolean operations =====
654:     0a100110      and      w16, w8, w16
658:     0a0a013c      and      w28, w9, w10
668:     0a100190      and      w16, w12, w16
6cc:     0a100170      and      w16, w11, w16
===== GPR additions =====
34:      9101c3f0      add      x16, sp, #0x70
7c:      0b0c039d      add      w29, w28, w12
80:      0b1103bd      add      w29, w29, w17
94:      0b10016b      add      w11, w11, w16
a0:      0b1001ef      add      w15, w15, w16
a4:      0b1e016b      add      w11, w11, w30
a8:      0b1d01ef      add      w15, w15, w29
ac:      0b1b016b      add      w11, w11, w27
Full disassembly: build/arm64_asm/sm3_arm64.disasm
wyxin@xfusion:~/sm3/sm3-project/sm3_arm$ 

```

图 4: ARM64 汇编后端的 NEON 与通用寄存器指令

5.2.3 性能测试方法

性能测试分别针对 Reference C、ARM64 NEON 和 ARM64 Assembly 三种实现进行。三种版本使用相同的完整 SM3 接口，每次计时均调用：

```
1 sm3_digest(message, message_size, digest);
```

因此，测试结果不仅包含单分组压缩函数，还包含初始化、消息分组、最终填充和摘要输出等完整处理过程。

测试消息采用确定性规则生成：

```
1 data[i] = (uint8_t)(i * 131U + 17U);
```

实验选择以下四种消息长度：

64, 1024, 16384, 1048576 byte.

这些测试点分别对应单分组短消息、多个分组组成的中等长度消息、16 KiB 消息和 1 MiB 长消息。对于每种消息长度，程序调整迭代次数，使处理的原始消息总量不少于 64 000 000 字节。

计时使用：

```
1 clock_gettime(CLOCK_MONOTONIC, &ts);
```

每种消息长度先执行 8 次预热，以减小首次执行、缓存状态和页面映射等因素的影响；随后独立执行 3 轮正式测试，并取其中耗时最短的一轮作为最终结果。

测试循环根据摘要的部分字节构造校验值，并写入 volatile 变量，避免编译器将摘要计算识别为无用代码并将其删除。

吞吐量按照下式计算：

$$\text{Throughput} = \frac{\text{Processed Bytes}}{\text{Elapsed Time} \times 10^6} \text{ MB/s.}$$

这里采用十进制单位，即：

$$1 \text{ MB} = 10^6 \text{ bytes.}$$

单位字节处理时间为：

$$\text{ns/B} = \frac{\text{Elapsed Time} \times 10^9}{\text{Processed Bytes}}.$$

NEON 和汇编版本相对于 Reference C 的加速比分别为：

$$\text{Speedup}_{\text{NEON}} = \frac{\text{Throughput}_{\text{NEON}}}{\text{Throughput}_{\text{Ref}}},$$

$$\text{Speedup}_{\text{ASM}} = \frac{\text{Throughput}_{\text{ASM}}}{\text{Throughput}_{\text{Ref}}}.$$

5.2.4 性能测试结果

ARM64 三种实现的代表性性能结果如表 9 所示。

表 9: ARM64 各实现性能对比

消息长度	Ref 吞吐量 (MB/s)	Ref 时间 (ns/B)	NEON 吞吐量 (MB/s)	NEON 时间 (ns/B)	ASM 吞吐量 (MB/s)	ASM 时间 (ns/B)	NEON 加速比	ASM 加速比
64 B	62.87	15.905	86.85	11.514	116.51	8.583	1.38×	1.85×
1024 B	121.12	8.256	170.50	5.865	232.59	4.299	1.41×	1.92×
16384 B	128.65	7.773	181.78	5.501	247.67	4.038	1.41×	1.93×
1048576 B	128.86	7.760	178.88	5.590	234.72	4.260	1.39×	1.82×

从表中可以看出，NEON 和手写汇编版本在四种消息长度下均快于 Reference C。NEON 版本的加速比稳定在 $1.38\times$ 至 $1.41\times$ 之间，说明利用 NEON 完成消息装载、字节序转换和部分并行计算能够有效降低单分组处理开销。

手写汇编版本进一步将加速比提高到 $1.82\times$ 至 $1.93\times$ 。在 16384 B 消息下，汇编版本取得最高吞吐率 247.67 MB/s，单位字节处理时间由 Reference C 的 7.773 ns/B 降低至 4.038 ns/B，加速比达到 $1.93\times$ 。

```
==== Reference implementation ====
64          62.87 MB/s
1024        121.12 MB/s
16384       128.65 MB/s
1048576     128.86 MB/s

==== ARM64 NEON implementation ====
64          86.85 MB/s
1024        170.50 MB/s
16384       181.78 MB/s
1048576     178.88 MB/s

==== ARM64 assembly implementation ====
64          116.51 MB/s
1024        232.59 MB/s
16384       247.67 MB/s
1048576     234.72 MB/s
```

图 5: ARM64 Reference、NEON 和汇编版本性能对比

5.2.5 加速效果分析

对于 64 B 短消息，Reference C 的吞吐率为 62.87 MB/s，NEON 版本达到 86.85 MB/s，汇编版本达到 116.51 MB/s。由于 64 字节原始消息正好占据一个完整分组，最

终填充还需要额外处理一个分组，因此初始化、填充、函数调用和摘要输出等固定开销占比较高。即便如此，汇编版本仍取得 $1.85\times$ 的加速，说明压缩核心的优化能够明显改善短消息性能。

随着消息长度增加，Reference C 的吞吐率逐渐稳定在约 129 MB/s。NEON 版本在 1024 B 和 16384 B 消息下分别达到 170.50 MB/s 和 181.78 MB/s，相对参考版本均获得约 $1.41\times$ 的提升。

手写汇编版本在 1024 B 和 16384 B 消息下分别达到 232.59 MB/s 和 247.67 MB/s，加速比分别为 $1.92\times$ 和 $1.93\times$ 。随着消息长度增加，压缩函数在总运行时间中的占比提高，16 字消息循环窗口、在线消息扩展和完全展开的轮函数能够更充分地发挥作用。

对于 1 MiB 消息，汇编版本吞吐率为 234.72 MB/s，相对于 Reference C 提升 $1.82\times$ 。该结果略低于 16384 B 消息下的最高吞吐率，但仍保持稳定的性能优势。该差异可能受到长时间运行过程中的处理器频率变化、系统调度和缓存状态等因素影响，仅凭当前计时结果不对具体微架构原因作进一步判断。

汇编版本相对于 NEON 版本的加速比分别为：

$$1.34\times, \quad 1.36\times, \quad 1.36\times, \quad 1.31\times.$$

这说明汇编版本的额外收益不仅来自 NEON 消息装载，还来自更精细的通用寄存器分配和数据流组织，主要包括：

- **16 字消息循环窗口。** W_0 至 W_{15} 始终保存在通用寄存器中，每生成一个新扩展字就覆盖已经失效的旧消息字，避免保存和反复访问完整的 $W[68]$ 数组。
- **消息扩展与压缩交错执行。** 前 12 轮直接使用初始消息字；从第 12 轮开始，每生成一个新的 W_{j+4} 就立即执行对应的第 j 轮，减少扩展结果的中间存储和再次读取。
- **派生消息字按需生成。** 每轮直接计算 $W'_j = W_j \oplus W_{j+4}$ ，不再保存完整的 $W'[64]$ 数组。
- **桶形移位操作数。** ARM64 的 `eor` 等逻辑指令允许源操作数附带 `ror`，从而将循环移位与异或合并到一条指令中。
- **轮函数完全展开。** 前 16 轮和后 48 轮分别采用不同汇编宏，所有轮次均在汇编期展开，消除了运行时循环判断、轮号分支和部分状态搬移。
- **独立加法链交错调度。** 压缩轮中更新 D 和 H 的两条加法链交错排列，使处理器能够在相互独立的指令之间提高指令级并行性。

总体而言，ARM64 NEON 版本已经能够获得约 $1.4\times$ 的稳定加速，而最终手写汇编版本在中长消息下将加速比进一步提高到约 $1.9\times$ 。实验结果说明，针对 SM3 的数据依赖特征合理划分 NEON 和通用寄存器任务，并结合循环窗口、在线消息扩展、轮函数展开和指令调度，能够在鲲鹏 920 平台上有效提高 SM3 软件实现的吞吐率。

6 总结

本实验完成了 SM3 的标准 C 实现，并分别面向 x86-64 和 ARM64 架构实现了 SIMD 寄存器与通用寄存器混合优化。公共 C 层负责消息初始化、分段更新、填充和摘要输出，架构相关后端主要优化单个消息分组的扩展与压缩过程。

在 x86 部分，实验采用 Multi-Buffer 多消息并行方案。AVX2 版本使用 YMM 寄存器的 8 个 32 bit lane 同时处理 8 条独立消息，AVX-512 版本则将并行宽度扩展到 16 路。正确性测试、OpenSSL 差分测试和 Sanitizer 检查均未发现错误，反汇编结果也确认程序实际生成了 YMM 和 ZMM 向量指令。由于实验主机不支持 AVX-512F，性能测试主要针对 AVX2 版本进行。

性能方面，AVX2 八路实现对不同消息长度均取得了显著提升。在 1024 B 消息的十轮重复测试中，吞吐率中位数由 188.39 MiB/s 提升至 1250.62 MiB/s，单位字节处理时间由 5.063 ns/B 降低至 0.763 ns/B，中位加速比达到 6.66 $\times$ 。在 64 B 至 1 MiB 的测试范围内，加速比总体保持在约 6.3 $\times$ 至 6.7 $\times$ 。

在 ARM64 部分，最终实现使用 NEON 完成完整消息分组的装载和字节序转换，并使用通用寄存器保存 16 字消息循环窗口以及 A 至 H 压缩状态。手写汇编进一步采用消息扩展与压缩交错执行、轮内计算 W'_j 、四轮寄存器角色轮换、64 轮完全展开和独立加法链交错调度等方法，以减少中间存储、状态搬移和运行时分支。Reference C、NEON 和 Assembly 三种实现均通过 24 项标准向量、填充边界和流式接口测试。

ARM64 NEON 版本在不同消息长度下取得了 1.38 $\times$ 至 1.41 $\times$ 的加速，手写汇编版本的加速比进一步提高到 1.82 $\times$ 至 1.93 $\times$ 。在 16384 B 消息下，汇编版本吞吐率达到 247.67 MB/s，单位字节处理时间由 Reference C 的 7.773 ns/B 降低至 4.038 ns/B，加速比达到 1.93 $\times$ 。

x86 与 ARM64 优化方案的主要差异如下：

- **x86 优化维度不同。** x86 通过 Multi-Buffer 方法并行处理多条独立消息，主要利用 YMM 或 ZMM 寄存器提高批量消息吞吐率。
- **ARM64 更侧重单消息内部优化。** ARM64 将适合并行的消息装载和字节序转换交给 NEON，将具有递推依赖的消息扩展和压缩关键路径交给通用寄存器。
- **寄存器协同方式不同。** x86 中 SIMD 寄存器承担多路状态和轮函数计算，通用寄存器主要负责地址、长度和批次调度；ARM64 中 NEON 与 GPR 分别承担输入处理和核心递推计算。
- **性能数据不直接跨平台比较。** 两部分使用不同处理器、操作系统环境和吞吐率单位，因此实验重点比较同一平台上优化版本相对于参考实现的提升，而不直接依据绝对吞吐率评价两种架构。

综合来看，本实验实现了面向两类处理器架构的 SM3 优化方案，并完成了正确性验证、反汇编检查和性能测试。实验结果表明，根据算法的数据依赖关系和目标处理器的寄存器、指令集及执行特点，选择合适的数据布局、并行维度和指令调度方式，能够显著提高 SM3 软件实现的执行效率。